

Sanskrit Retrieval-Augmented Generation (RAG) System

This project implements a Retrieval-Augmented Generation (RAG) system for Sanskrit documents. The system allows users to ask questions from Sanskrit PDF files and generates meaningful answers using semantic retrieval and Large Language Models (LLMs). The project was developed using LangChain, ChromaDB, HuggingFace embeddings, and HuggingFace chat models. The complete system works on CPU and follows a modular RAG architecture. The main goal of the project is to improve answer generation by retrieving relevant Sanskrit context before generating responses. Instead of relying only on the knowledge already stored inside the LLM, the system first searches the uploaded Sanskrit documents and then generates answers from the retrieved content.

1. System Architecture and Flow

The system follows a step-by-step RAG pipeline. Each module performs a specific task. This modular structure makes the project easy to understand, maintain, and improve. The complete workflow is shown below:

PDF Documents ↓ PyPDFLoader ↓ RecursiveCharacterTextSplitter ↓ Text Chunks ↓ HuggingFace Embeddings ↓ ChromaDB Vector Database ↓ Similarity Retriever ↓ Prompt Construction ↓ ChatHuggingFace Model ↓ Generated Answer

Component	Purpose
PyPDFLoader	Loads Sanskrit PDF documents
RecursiveCharacterTextSplitter	Splits large text into smaller chunks
Embedding Model	Converts text into numerical vectors
ChromaDB	Stores embeddings and document chunks
Retriever	Finds relevant chunks using semantic similarity
ChatHuggingFace	Generates final answer using retrieved context
Streamlit	Provides user interface

2. Details of Sanskrit Documents Used

The project uses Sanskrit PDF documents containing stories, moral lessons, devotional content, and Sanskrit prose passages. The documents include: • Kalidasa stories • Sanskrit moral stories • Devotional teachings • Traditional Sanskrit narratives • Sanskrit explanations and prose The PDF files are stored inside the data folder. The system dynamically reads all PDF files using PyPDFLoader. Each page inside the PDF becomes a LangChain Document object. This design allows the system to easily support multiple Sanskrit documents. New PDF files can be added to the data folder without changing the code.

3. Preprocessing Pipeline for Sanskrit Documents

The preprocessing stage is one of the most important parts of the RAG pipeline. Its purpose is to prepare the Sanskrit documents for semantic retrieval. The preprocessing pipeline contains multiple stages:

- Document Loading using PyPDFLoader
- Text Splitting using RecursiveCharacterTextSplitter
- Chunk Creation with overlap
- Embedding Generation
- Vector Storage in ChromaDB

Document Loading:

The Sanskrit PDF files are loaded page-by-page using PyPDFLoader. Each page is converted into a structured LangChain Document object.

Text Splitting:

The loaded documents are split into smaller chunks using RecursiveCharacterTextSplitter. This splitter preserves sentence continuity and semantic meaning better than fixed splitting. The following configuration was used: • chunk_size = 500 • chunk_overlap = 50 Chunk overlap helps preserve context between adjacent chunks.

Embedding Generation:

After chunking, the text chunks are converted into vector embeddings using the multilingual model: sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2 Embeddings are numerical representations of text meaning. Semantically similar sentences generate similar vectors.

Vector Storage:

The generated embeddings are stored inside ChromaDB. ChromaDB allows fast semantic similarity search over document chunks.

4. Retrieval and Generation Mechanisms

The RAG system combines retrieval and generation into a single workflow. **Step 1 — User Query**
The user enters a question through the Streamlit interface.

Step 2 — Query Embedding

The user query is converted into an embedding vector using the same embedding model.

Step 3 — Similarity Search

The retriever compares the query vector with stored document vectors inside ChromaDB. The retriever uses: • search_type = similarity • k = 3 This means the top 3 most relevant chunks are retrieved.

Step 4 — Prompt Construction

The retrieved chunks are combined into a prompt along with the user question. Example: Context: (retrieved Sanskrit chunks) Question: (user query) Answer:

Step 5 — Answer Generation

The prompt is sent to the HuggingFace chat model. The model generates a context-aware answer using only the retrieved information. The following parameters were used: • temperature = 0.3 • max_new_tokens = 400 A lower temperature value helps reduce hallucinations and improves factual consistency.

5. Performance Observations

The system performance was evaluated using: • Latency • Retrieval relevance • CPU usage • RAM usage **Latency:** Latency measures how much time the system takes to generate answers. Observed timings: • Document Loading: 1–2 seconds • Embedding Generation: 3–8 seconds • Retrieval + Generation: 2–5 seconds The latency depends on: • number of documents • document size • retrieved context length • internet speed for hosted LLM

Retrieval Accuracy:

The similarity retriever successfully returned relevant Sanskrit chunks for most queries. Example test queries: • Who was Kalidasa? • ■■■■ ■■■ ■■■■■■■■ ■■■■■ ? • What is the moral of the story? The multilingual embedding model improved retrieval quality for both Sanskrit and English questions.

Resource Usage:

The project successfully ran on CPU without GPU acceleration. Observed usage: • CPU Usage: Moderate • RAM Usage: Moderate • GPU Usage: None The lightweight embedding model and ChromaDB made the system efficient for local execution.

6. Challenges Faced

During development, several challenges were encountered: • Sanskrit text handling and semantic retrieval • Proper chunk sizing for better retrieval • Managing Streamlit caching issues • Path handling for PDF documents • Balancing retrieval quality and response speed These challenges were solved using modular architecture and LangChain utilities.

7. Conclusion

The Sanskrit RAG System successfully implemented document ingestion, semantic retrieval, and context-aware answer generation using open-source technologies. The project demonstrates how Retrieval-Augmented Generation can improve answer quality by combining semantic search with Large Language Models. The modular structure makes the system scalable and easy to improve in future versions. Overall, the system achieved: • Efficient CPU-based execution • Relevant Sanskrit retrieval • Context-aware answer generation • Beginner-friendly modular architecture

8. Future Improvements

Future improvements may include: • Hybrid retrieval methods • Better multilingual LLMs • Sanskrit OCR support • Chat history memory • Reranking systems • Deployment on cloud infrastructure